

高速 ACT 战斗系统原型 · 战斗设计案

UE 5.8 · 纯蓝图 · 个人项目 · 2026.08 至今

吴云柯 (Wink) | 15858252358@163.com | winnnnk.github.io

一、这个 demo 是什么

一个白模竞技场里的高速 ACT 战斗系统，玩家侧完整垂直切片：三段轻击、按资源分段释放的重击、闪避。UE 5.8，纯蓝图，不使用 GAS。

所有时序参数集中在一张 23 列的帧数据表里。取消矩阵是这张表的视图，本身不存数值，调整任何一个窗口都只要改表，不用动蓝图。

当前状态

更新于 2026-08-31。

状态	内容
已实现	输入缓冲与优先级仲裁、跨状态缓冲保留策略
已实现	取消矩阵仲裁、招式执行、球体扫描判定与单次挥砍去重
已实现	「决」的积累与分段释放仲裁、帧表运行时校验
已实现	玩家受击链路：血量、伤害、24 帧硬直、死亡、缓冲保留规则
已实现	完美闪避全套：帧表驱动的无敌帧、0.30x 时缓 0.75 秒（8 帧插值退出）、决 +33、1 秒反击窗口
已实现	专属反击招式 Counter_01：完闪窗口内轻击自动变招、突进落位、必破韧
已实现	攻击软索敌：按 I_Damageable 找最近目标，朝目标转向与突进
已实现	Boss 单次打击 Boss_Smash_A，判定窗口帧 20-23（11 次挥击实测）
部分实现	Boss 行为：目前是「冷却到了就挥一刀」的最简循环，状态机未做
未实现	Boss 受击 ：I_Damageable 已挂，TakeCombatHit 仍是空实现，Boss 不掉血、无韧性条
已定义	Boss 一阶段其余招式（5-6 招）、四类可应对标记、决策权重表
已定义	二元护甲门槛与破韧后的处决窗口
未实现	反馈链的屏震 / FOV / 音效 / 特效（目前只有顿帧）
未实现	单目标锁定、四方向闪避位移、战斗姿态 locomotion、正式 UI

本文里凡是标「已定义」的部分，规格是完整的，只是还没有进引擎。

一个攻防回合目前差最后一环：玩家能被打、能完美闪避、能反击，判定也能命中 Boss，但 Boss 不会掉血。补上 Boss 的血量与韧性即闭环，那是下一步的第一件事。

为什么不用 GAS

战斗策划岗要证明的是六件事：能不能把手感拆成起手 / 判定 / 收招 / 取消窗口 / 位移 / 无敌帧这些可量化参数；懂不懂输入缓冲、取消规则、优先级仲裁；会不会设计反馈链的时序分级；Boss 招式的风险收益与读招提示；TTK、DPS、韧性、伤害曲线能不能复算；能不能自己在引擎里把设计验证出来。

GAS 只服务于第六项里偏程序的一小部分。在蓝图零基础的前提下，三个月学 GAS 会吃掉本该花在前五项上的时间，最后交出一个架构对但打起来不好玩的 demo。GameplayTags 是独立插件、不依赖 GAS，正常使用。

二、设计理念

高速 ACT 战斗里最重要的是玩家角色和 Boss 之间的交互。

我认为魂系那种以体力槽为核心的攻防，本质上是回合制——你打完我打，中间由体力决定谁能动。而动作游戏的主基调应该是见招拆招、手眼协调、互相进攻，以及夹在其中的一点点策略。

所以这个 demo 鼓励玩家和 Boss 对着打。面对 Boss 的攻击，玩家可以闪避也可以格挡，完美闪避和完美格挡都给很高的回报。Boss 的韧性条会在一段时间不受攻击后随时间回升，这是为了让玩家在观察 Boss 出招的同时不能完全不出手。

给勇敢的玩家应得的奖励，在玩家没有犯错的时候不要惩罚他——这是我的设计理念。

其中「完美格挡」与「韧性条随时间回升」目前列为待做项，尚未进入实现。理念可以先于实现确定；下面每一条规则的取舍，都会回到这一节来解释。

三、范围

核心动词集

动词	优先级	说明
移动 + 单目标锁定	P0	进入战斗自动锁住 Boss，相机看向它、角色面向它
地面三段普攻	P0	每段独立 Montage Section，段间取消窗口可调
重击 / 决释放	P0	轻击、完闪与低效蓄力积累「决」；松开重击按当前完整段数释放，满决可破甲
闪避	P0	起始无敌帧、可取消普攻任意段、有位移和方向
完美闪避	P0	无敌帧内被有效攻击判定命中 → 时缓 + 镜头推进 + 限时反击窗口
完美格挡 / 弹反	待做	短窗口格挡成功 → 弹开 Boss + 处决机会

明确不做

空中连段与浮空（需要浮空受击、落地、重力曲线、空中相机一整套，成本 40 小时起）；第二名角色、多敌人、小怪 AI；跑图关卡与训练场；完整 GAS、C++、行为树；开放世界、剧情、装备、存档、联网。

范围收窄的目的是把时间全部压在战斗手感上。任何新想法都要说明它挤掉哪个里程碑。

四、取消规则

玩家在任何一个瞬间按下一个键会发生什么，是高速 ACT 里他唯一真正感知到的东西。他说这游戏跟手或者黏，说的就是这张表。

4.1 三条原则

整张矩阵由这三条推导出来，不是逐格填的。

#	原则	理由
R1	起手是承诺，判定之后才有出路。 任何攻击的 DodgeCancelStart = 它的 ActiveStart	如果起手就能闪，攻击不需要付代价；如果要等收招才能闪，玩家不敢起手。切在判定开始，风险与出路各占一半。按下攻击这个决定本身不该被惩罚，读错时机才该
R2	打完才能接，空挥要等。 ComboCancelStart 开在 ActiveEnd 之后 2-4 帧	空挥时这几帧就是代价，打中时几乎感觉不到。连段的顺滑是打中之后才给的
R3	闪避优先级永远最高，无例外。 任何状态下只要闪避窗口开着，闪避输入压过一切	这条一旦开例外（比如「重击不可被闪避取消」），玩家会干脆不用那个招式。玩家想活下来的时候，系统不该拦他

ComboCancelStart 的偏移量按招式分量给出承诺代价：轻击 +3、快重击 +6、决斩 +12。

4.2 取消矩阵

行 = 当前正在做什么，列 = 玩家按了什么，格子 = 从第几帧起这个输入会被响应。F0 = 立即；X = 该状态下永不响应，输入进缓冲等状态结束。

当前状态	总帧	→ 轻击	→ 重击	→ 闪避	→ 移动
Idle / Locomotion	—	F0	F0	F0	F0
Light_1	90	F16 → Light_2	F16 → 重击仲裁	F11	仅转向
Light_2	78	F11 → Light_3	F11 → 重击仲裁	F3	仅转向
Light_3	83	F43 → Light_1 (循环)	F43 → 重击仲裁	F9	仅转向
ResolveCharging (积累决)	不定	X	松手 → 重击仲裁	F1 (取消且不释放)	仅转向
Heavy_Quick	103	F22 → Light_1	X	F11	仅转向
Heavy_Release_1/2	79	F19 → Light_1	X	F2	仅转向
Heavy_Release_3	139	F75 → Light_1	X	F3	仅转向
Dodge	80	F16 → Light_1	F16 → 重击仲裁	F22	F16
HitReact_Light	24	F14	F14	F6	F14
HitReact_Heavy	60	X	X	F42 (起身闪避)	F48
Dead	—	X	X	X	X

上表所有帧号都来自运行时实测，不是设计预期值。取数方法见第八节。

「仅转向」的实现：角色 `bUseControllerRotationYaw = true`，朝向跟随摄像机，与移动输入是两路。把位移清零，转向自动保留。解锁帧取 `ComboCancelStart` —— 取消窗口打开等于承诺结束，该把控制权还给玩家，这两件事本来就是同一件事，不该是两个数。

4.3 四处刻意的不对称

矩阵里有四个地方我故意没有对称。这一节比矩阵本身更重要，它要证明的不是我会做取消系统，而是我知道每一个数字在引导玩家做什么。

① 闪避接攻击 F16，比闪避接闪避 F22 早 6 帧

闪完接攻击比闪完再闪快 6 帧，玩家自然会选前者。如果反过来，连续闪避更划算，高速 ACT 会退化成一路滚到 Boss 背后再打。这 6 帧决定了这套战斗奖励进攻还是奖励逃跑，对应设计理念里的「鼓励玩家与 Boss 对攻」。

② 重击的轻击取消 (F19–F22) 远早于收招结束 (F79–F103)

重击留了 60–81 帧的提前脱离空间。它的代价由空挥承担：没打中就吃满收招。打中了却还要站在原地挨一套，等于惩罚一个做对了的玩家，重击就没人愿意用。

③ 受轻击硬直里，闪避取消 F6 远早于攻击取消 F14

被打的时候能更快闪，但不能更快打。受击硬直里玩家最需要的是脱身手段，F6 就把它给出去；反打要多等 8 帧。这是 R3 在受击状态下的落实。

④ 轻击三段的取消帧 F43，是一二段 (F16 / F11) 的三倍以上

我最初对三段轻击套用了同一条取消规则。试玩时发现第三段刚砍完就能被第一段打断，动画被掐掉，而且循环一次没有任何代价。

归因之后才想明白：前两段的取消是「连接」，第三段的取消是「循环」。一二段截掉收招看不出来，后一刀的起手接得上前一刀的收势；三段的收势和第一段的起手对不上，早取消就会看到第三刀被生生掐断。两者不该用同一条规则。

F43 是挥砍视觉收完、开始回站姿的那一帧——让玩家看完这一刀，但不必看完回身。

代价是 `ComboCancelStart` 同时管轻击与重击取消，所以三段接决斩也一并推到 F43。若这条路径实测确实需要，再拆出独立的 `ComboLoopCancelStart` 列，只锁「循环回第一段」这一条边。`DodgeCancelStart` 保持 F9 不变，R3 没有例外。

4.4 唯一的例外：完美闪避特权窗口

触发条件：`Dodge` 的 F4–F15 无敌帧区间内被 Boss 有效攻击判定命中。窗口期 60 真实帧（1 秒）内，矩阵被三条覆盖：

覆盖项	值	说明
当前 Dodge 的所有取消帧	提前到 F+2	完闪成功后两帧就能反打，这个突然自由的感觉本身就是奖励
窗口内攻击的 <code>StartupFrames</code>	× 0.5 向下取整	Light_1 由 8 降到 4
窗口内攻击的 <code>bForcePoiseBreak</code>	true	一击必破韧

窗口内 R1 / R2 依然生效，只是帧号被压缩。没有任何时候是完全无规则的——一旦给出「窗口内随便取消」，玩家会发现最优解是完闪后无脑连打，读招的意义就没了。

五、输入缓冲与仲裁

5.1 缓冲参数

参数	值	调它影响什么
BufferDurationFrames	12 帧 (200ms)	调大 → 更宽容、连段更容易打出来，但会出现「我明明没按第三下」的误触发；调小 → 更精确更硬核，但新手打不出连段
BufferSlotCount	1 (只存最后一个有效输入)	存多个会导致按错的键在两秒后突然生效
写入时机	按下瞬间，且当前状态不能立即响应应该输入	
过期判定	用真实帧计时，不受 Hitstop 影响	Hitstop 期间玩家还在按键，缓冲必须照常倒计时

12 帧的取值区间：向上受「100ms 内的延迟不被感知为卡」约束，缓冲太长会让玩家感到输入被延迟执行；向下受最短取消窗口宽度约束，Light_1 的窗口从 F16 到 F32 共 16 帧，缓冲若短于窗口一半，玩家提前按就会丢。

5.2 优先级仲裁

输入	优先级	覆盖规则
闪避	3 (最高)	无条件覆盖缓冲中的任何输入
重击	2	覆盖缓冲中的轻击；不覆盖缓冲中的闪避
轻击	1	只覆盖缓冲中的轻击（刷新时间戳）
移动	0	不进缓冲，实时处理

高优先级输入覆盖低优先级缓冲，低优先级不覆盖高优先级。这直接实现了 R3：玩家在慌乱中乱按时，闪避一定打得出来。

因为槽位是 1，同一时刻缓冲里只有一个输入，同帧多窗口同时开放时结构上不可能有冲突。

5.3 跨状态的缓冲保留策略

状态切换时缓冲怎么处理，是最容易出 bug 也最容易被追问的地方。

状态转换	攻击缓冲	闪避缓冲	理由
攻击 → 攻击（连段推进）	消费并清空	保留	正常路径
任意 → HitReact（被打了）	清空	保留	被打断时「我刚才想打的那一下」已经没意义了；但「我想闪」必须留着，让玩家能在受击硬直结束的第一帧闪出去
任意 → Dodge	清空	消费并清空	
Dodge → 完美闪避窗口	保留	清空	完闪触发瞬间玩家往往已经按下攻击了，这一下必须打出来，否则完闪奖励感受不到
任意 → Dead	全部清空	全部清空	

5.4 每 Tick 的仲裁流程

每 Tick:

- ├ 1. 缓冲过期检查
 - | $\text{RealFrame} - \text{BufferedAtFrame} > 12 \rightarrow$ 清空缓冲
- ├ 2. 新输入处理（本 Tick 有按键时）
 - | ─ 当前状态能立即响应？ 一是 $\rightarrow$ 直接执行，不写缓冲
 - | ─ 否 $\rightarrow$ 优先级 $\geq$ 缓冲中的？ 一是 $\rightarrow$ 写入缓冲（含 RealFrame 时间戳）
 - | ─ 否 $\rightarrow$ 丢弃
- ├ 3. 缓冲消费检查（缓冲非空时）
 - | ─ $\text{CurrentFrame} = \text{FloorToInt}(\text{MontagePosition} \times 60)$
 - | ─ 查帧表取当前招式的 ComboCancelStart / DodgeCancelStart
 - | ─ 应用完美闪避 override（若生效）
 - | ─ $\text{CurrentFrame} \geq$ 对应窗口起始帧？ 一是 $\rightarrow$ 执行 + 清空缓冲
- └ 4. 状态自然结束
 - | $\text{CurrentFrame} \geq \text{TotalFrames} \rightarrow$ 回 Locomotion（缓冲保留，下一 Tick 由 3 消费）

第 4 步不清空缓冲，这样「我在收招末尾按的那一下」会在回到 Locomotion 的下一帧立刻打出来。少了这条，玩家会觉得招式之间有个吞输入的黑洞。

两个实现约束：缓冲时间戳用真实帧，不用 MontagePosition（Hitstop 会让它停）；CurrentFrame 用 $\text{MontagePosition} \times 60$ ，不用 Tick 累加（Hitstop 会让计数偏移）。

六、「决」资源系统

6.1 为什么要做一个资源系统

战斗的交互不仅限于攻防两端，资源的获取与消耗同样构成交互的一部分——玩家需要判断何时积累、在哪一档消耗、是否承担风险等待更高一档。基于这一判断，本 demo 设计并实现了「决」资源系统。

6.2 这个资源不限制玩家的行动

该系统不采用体力槽式的行动限制。攻击、闪避、反击等基础行动不消耗资源，任何时刻均可执行；资源由行动产生，并另行提供消耗途径。这一结构与传统 ACT RPG 及二次元手游构建战斗循环的方式一致。

「决」是本 demo 中唯一的可消耗资源，作用于单次攻击的分量，不作用于行动的可用性。这一取舍与第二节的设计理念一致：限制玩家在未犯错时可以做出的行动，本身即是一种惩罚。

6.3 结构与产出路径

上限 99，分 3 档，每档 33。分档而非连续条，是为了给玩家明确的中间目标与「是否再攒一档」的决策锚点。

决值	解锁
33	初始强化重击
66	更高一阶伤害
99 (满)	满决重击，破除 Boss 护甲的唯一手段

三条产出路径：轻击命中、按住重击蓄力、闪避反击。完整产出表如下。

来源	决	说明
Light_1 命中	+12	
Light_2 命中	+14	
Light_3 命中	+20	递增
Heavy_Quick 命中	+10	无决时的保底重击也给一点，避免无决状态下按重击纯亏
蓄力（按住重击键）	+0.35 / 帧	效率为轻击链的 45.7%
完美闪避成功	+33 (一整档)	
完闪窗口内首次命中	+20	反击追加
受击	-15	使积累本身带有风险

6.4 数值的反解

数值不是先填后调的。先写下想要的体验，再让每一项的收益去凑这个等式。

完美闪避(+33) + 窗口内首次命中(+20) + 一轮完整三段连(12+14+20=46) = 99 = 满决

即：读中一次招，加上打出一套完整连段，正好破甲。

三条路径的实际时长（占空比 40%）：

路径	输出帧	真实时长
纯输出（0 次完闪）	129	5.4s
1 次完闪 + 反击命中	60	2.5s
2 次完闪 + 反击命中	0	立即满决
纯蓄力（够不到时兜底）	283	~11.8s

一次完美闪避使破甲耗时减半。玩家无需任何 UI 说明即可察觉读招快于站桩，这是设计理念中「给高风险动作高回报」的直接落点。

6.5 每一档的取值依据

轻击三段产出递增（12 / 14 / 20），完整连段的收益高于中断重来，无需 UI 引导即可被玩家察觉。

三档释放的伤害与决值之比呈超线性，积累越充分，单位资源的伤害转化率越高：

档位	伤害	破韧	伤害 / 决	破甲
1 档	70	30	2.12	X
2 档	150	65	2.27	X
3 档（满决）	250	110	2.53	✓

三档共用同一个 Montage，靠反馈档与 VFX 尺寸区分威力，无额外动作成本。3 档破韧 110 大于 Boss 韧性 100，满决释放必定破韧。

蓄力效率取轻击链的 45.7%，定位为无法接近目标时的替代路径；若两者效率接近，远距离蓄力将成为最优解，战斗节奏随之瓦解。

闪避反击单次产出 33，等于一整档，使读招成为三条产出路径中效率最高的一条。

仲裁规则：正常态下决 < 33 出 Heavy_Quick，决 ≥ 33 按完整档数选择释放技；护甲态下决 < 99 出 Heavy_Quick 且不消耗决，决 = 99 出满决重击。释放技开始执行时才扣除资源，输入仍在缓冲或 Montage 播放失败时不得预扣。

6.6 Boss 护甲与满决破甲

Boss 血量降至 50% 以下时进入护甲状态。该状态下玩家造成伤害将锁定 Boss 韧性条，Boss 获得 90% 减伤，仅满决重击可破除护甲，破甲后进入处决窗口。

护甲机制成立的前置条件：护甲期轻击伤害衰减至 10%，但决值照常积累。若护甲期轻击既不产出伤害也不产出资源，玩家将无法积累破甲所需资源，战斗进入死锁。

这条前置条件成立之后，护甲期改变了轻击这个动作的意义：一阶段按轻击是为了造成伤害，护甲期按轻击是为了积累破甲资源。同一个按键在两个阶段承担不同职能，而不是引入一个新招式。

护甲态下决不满时按重击出 Heavy_Quick 且不消耗决。允许 1 / 2 档也能释放并白白浪费会更硬核，但代价是一次误操作损失全部积累，而误操作在高速战斗中必然发生。惩罚应当来自读错招，不应来自手滑。

6.7 破甲之后：处决窗口

破韧成立 → 【倒地】 → 【处决窗口】约 120 帧（2 秒）

- Boss 挂易伤 buff
- Boss 韧性条锁定（防止窗口内二次破韧形成无限循环）
- 表现：跪姿 + 材质变红 + 处决提示
 - └─ 玩家按处决 → 固定高伤害 + 配对演出 → 回到正常态
 - └─ 窗口耗尽 → 起身，易伤与锁条清除 → 回到正常态

120 帧是这套设计的核心参数。它必须短到只够做一件事：够打一套三段轻击，或者够做一次处决，不够两个都做。

选择	得到	失去
处决	高额固定伤害；安全收场，不吃起身反击	不攒决，下一轮从零开始
打输出	易伤加成下的三段连，照常攒决 $12+14+20 = 46$	伤害更低；要承担起身反击的风险

这个选择的真实含义是「我要现在的伤害，还是下一轮的资源」。它复用 6.4 的核心等式（46 决 = 一套完整连段），不需要任何新 UI、新数字、新教学。玩家不理解这个权衡也能玩，会玩的人能算出来。

如果窗口长到能先打输出再赶在关闭前处决，那么「两者皆要」严格优于其余选项，它就不再是博弈，而是一个计时器操作测试，最优解退化为「等 Boss 倒 → 倾泻输出 → 收尾」。这正是要避开的失败模式：**RPG 感不来自机制本身，来自最优解不需要交互。**

三条硬约束：状态靠 Boss 的身体传达（跪姿、材质、呼吸起伏），不靠 UI 图标和数字；窗口短到只能反应，不能规划；处决有前摇、有距离要求，不是按了必中的安全按钮。

七、Boss 规格（单次打击已实现，其余为下一里程碑）

7.1 一阶段：四类可应对

每招都标注可应对类型，这是「高频交互」的核心，玩家要在极短前摇内读出这是哪一类。

类型	视觉提示	玩家正确应对	招式数
可弹反	起手蓝光	弹反 → 处决机会	2
只能闪	起手红光	完美闪避 → 反击窗口	2
必须跑位	地面预警圈	拉开距离	1
追击 / 连段	无提示，多段	判断段数后反打	1

Boss AI 不用行为树，用蓝图状态机加一张决策表（列：距离区间 / 玩家状态 / 冷却 / 权重）。行为树对策划是黑盒，调起来慢；数据表可以直接摊开说明决策逻辑是怎么设计的。

7.2 二阶段：护甲是门槛，不是血条

HP ≤ 50% → 阶段切换演出

↓

【护甲态】二元状态，没有层数、没有护甲条

- 轻击 / Heavy_Quick 伤害 × 0.1
- 轻击照常攒决 ← 这条不成立整个机制死锁
- 韧性积累锁定为 0
- 决 < 99 时按重击 → Heavy_Quick，不消耗决
- 决 = 99 时按重击 → 破甲斩
- Boss 出招频率 +30%

↓ 满决释放命中

【破甲】→ 立即触发【处决】固定伤害 + 简版演出

↓

【正常态】完全回到一阶段规则

↓ HP ≤ 25%

【护甲态 #2】→【处决 #2】= 击杀演出

7.3 两条生死线

「护甲期轻击照常攒决」是死锁与否的分界。护甲期轻击若既不给伤害也不给决，玩家没有任何手段积累破甲资源，战斗直接死锁。这条成立之后有一个后果：一阶段玩家按轻击是为了打伤害，护甲期按轻击是为了充能——伤害只有 10%，但决照常涨。同一个按键、同一个动作，意义完全变了。用一个机制改写了整场战斗的语法，而不是加了一个新招式。

「不满决不消耗」是容错设计。护甲态下决不满时按重击，出 Heavy_Quick 且不消耗决。让 1/2 段也能放、放了白费会更硬核，但代价是玩家一次误操作损失全部积累，而误操作在高速战斗里必然发生。惩罚应该来自读错招，不该来自手滑。

这么做还有一个额外收益：护甲态下重击键的行为随决条变化。决不满时它是个平淡的保底重击，决条填满的那一瞬间，同一个键变成破甲斩。玩家会盯着决条等那一刻，这个等待本身就是张力。

7.4 教学结构：三层递进

阶段	玩家学到什么	系统怎么教
一阶段	攒满了再放，比乱放划算	超线性收益 (2.12 → 2.53) + 满决必破韧。没有强制，玩家自己算出来
二阶段护甲态	必须攒满才能破甲	把「可以选择攒满」变成「必须攒满」。同一个机制，从优势选项升级为生存条件
护甲态里的完闪	读招比站桩快一倍	一次完闪让破甲时间从 5.4s 降到 2.5s。不用 UI 解释，玩家自己感觉得到

玩家不需要任何教程。他在一阶段凭直觉摸索出来的东西，在二阶段成了唯一出路；他在二阶段被逼着练的读招，正好是这个 demo 想让他学会的核心技能。二阶段加深的是玩家已有的理解，而不是增加要记的信息量。

7.5 时长账

项	值
Boss 总 HP	5000
一阶段 (2500)	~40s
二阶段 (2500)	~31s
无死亡通关	~72s
含 1-2 次失误重来	~2.5-3 分钟

一阶段不上护甲，资源循环在这里是更优解而非必需品：纯轻击链有效 DPS 37.5，攒满释放 62.3，高 66%。这个差距够大到玩家会主动用，又不至于让不用的人打不过。

八、帧数据：单一真相源

8.1 七条推导原则

表里所有推断值都由这七条推出来。改原则则全表重算，不改原则只做单格微调。

#	原则	约束
P1	响应性优先：按下到画面出现第一帧攻击动画 ≤ 3 帧 (50ms)	起手帧再长，预备姿态必须 F1 就动
P2	普攻起手 ≤ 10 帧 (167ms)	Light_1 / Light_2 起手 ≤ 10
P3	三段打满约 90–110 帧 (1.5–1.8s)，这是高速 ACT 的体感门槛	三段连打总时长受限
P4	段间取消窗口开在判定结束后 2–3 帧	<code>ComboCancelStart = ActiveEnd + 2~3</code>
P5	闪避取消从判定开始帧起开放	<code>DodgeCancelStart = ActiveStart</code>
P6	重击 = 2× 普攻起手，换 2× 伤害 + 3× 破韧 + 长收招	Heavy 系起手 ≥ 18
P7	收招是唯一的惩罚来源	<code>TotalFrames - ActiveEnd</code> ，重击 ≥ 28

8.2 五条不变式（每次启动时运行时校验）

#	不变式
①	无判定招式 (<code>MoveCategory == Dodge</code>) 一律填 <code>ActiveStart = ActiveEnd = -1</code> ，跳过第 ② 条，只校验 <code>0 ≤ StartupFrames < TotalFrames</code>
②	<code>0 ≤ StartupFrames < ActiveStart ≤ ActiveEnd < TotalFrames</code> （仅对有判定的招式）
③	<code>ComboCancelStart == -1</code> 或 <code>ActiveEnd < ComboCancelStart < TotalFrames</code>
④	<code>DodgeCancelStart == -1</code> 或 <code>0 < DodgeCancelStart ≤ TotalFrames</code>
⑤	<code>IframeStart == -1</code> 或 <code>StartupFrames ≤ IframeStart ≤ IframeEnd < TotalFrames</code>

校验函数在每次开始游戏时跑一遍，违规时报出「哪一行 / 违反第几条 / 涉及的数各是多少」，而不是只说一句数据错误。

其中第 ① 条是补上去的：没有这条例外，`Dodge` 行会让每次闪避都触发报错。这类「合法的特例」如果不显式写进校验，校验本身就会变成噪音，然后被忽略。

8.3 帧数据是实测的

引擎的 API 读不到 Animation Notify 的起止帧，读和写两个方向都不支持。判定窗口是这套系统里最关键的一组数字，而我拿不到。

解决办法是让判定条自己报数：运行时把当前帧号写进日志，再把日志读回来。实测值与编辑器里显示的完全一致。

8.4 回填之后发现的问题

轻击二段的取消帧原本写的是 F18，而它的判定第 8 帧就结束了。中间 10 帧是纯死时间，正是试玩时「不跟手」的直接来源。一直用设计预期值的话，这个问题不会浮出来。

回填的同时还发现，最初那批帧号是在动作包到手之前拍的设计预期值，与实际动画长度差距很大。全表按实测重算之后，取消矩阵才第一次和手感对上。

九、一次判断失误的复盘

角色的 locomotion 还是引擎模板的徒手动画，而攻击动作是持剑姿态。每次收招都会在 6 帧内被拽回徒手站姿——角色演了 54 帧回到战斗站姿，最后 6 帧全部作废。

我一度误判了它。当时提出的验证方法是「看角色是不是握着剑」，但剑是挂在手骨上的静态网格，播任何动画都在手里，这个观察不可能推翻任何假设。后来查资产依赖才确认，引用的动画全部在 Unarmed/ 目录下。

之后我给自己定了一条：先问什么结果能推翻这个猜想，再去做测试。

这件事还带出一个更普遍的现象：手感问题常常在另一个改动之后才变得可见。第三段轻击「缺回手动作」这个缺口，在 ComboCancelStart 还是 F16 的时候根本看不出来，因为收招几乎不播；推到 F43 之后收招有 30 帧真正播出来，缺口才浮现。当时定位不了，不是判断错了，而是参数把症状盖住了。

十、借鉴谱系

借了什么、改了什么、为什么改。

机制	借鉴自	我改了什么
轻击生产 / 重击消费的资源循环	黑神话：悟空（棍势）	保留结构。这是地基，不必回避
满资源才能破甲、一击定生死	只狼（躯干条 → 忍杀）	只狼攒的是打在对方身上的架势；我攒的是玩家自己的决意，主动权在玩家，不在 Boss 的容错
防御成功直接转化为进攻资源	贝优妮塔 / 忍者龙剑传	它们给的是时间，我给的是资源数量。资源可累积、可跨回合携带
护甲期打不动血但推进另一条进度	怪物猎人（部位破坏）	怪猎的部位破坏是可选支线，我的是唯一主线
破甲 → 处决 → 回归正常出伤	只狼（忍杀后继续打）	

资源循环的壳来自黑神话悟空，临界点一击的内核来自只狼，防御成功转化为进攻资源的接口来自贝优妮塔与忍者龙剑传。

十一、里程碑与止损

11.1 里程碑序列

不写日期，写「做完什么算完成」。

阶段	完成标志
攻击闭环	打木桩：有判定、有掉血、有顿帧，Debug HUD 显示当前状态与帧号
连段与取消	慢按出单段、快按连满三段；重击按决档正确释放；乱按不卡死；闪避能取消任意段
完美闪避与 Boss 骨架	能与 Boss 打完一个完整攻防回合，完美闪避手感成立
Boss 一阶段完成	能从满血打到 50%，四类招式都能被正确应对
二阶段与演出	从头打到死；满决能稳定破甲；两阶段体感差异明显
打磨与交付	陌生人能在 3 分钟内打完

当前进度：前两项已完成，第三项进行中。

11.2 止损梯度

触发条件：任意里程碑连续延期两周，立即执行下一级，不临时讨论。

- 砍弹反，只保留完美闪避
- 砍 Boss 二阶段，改为血量 50% 后的狂暴提速（纯数值驱动，不重做动作）
- 砍处决与进阶演出，保留顿帧和镜头推进
- 砍可玩 Windows Build，只交视频、设计案与帧数据表
- 砍 Boss 招式数，一阶段压到 4 招

永远不砍的四样：三段普攻的取消手感、输入缓冲、完美闪避、反馈链分级。这四项是整个 demo 说服力的来源，砍掉任何一项，剩下的部分都不再能说明问题。

顺序预先写死而不是到时候再讨论，是因为进度落后时判断会偏向「再撑一下」，而那通常是最坏的选择。范围收缩必须在压力出现之前就决定好。

11.3 明确不做

空中连段与浮空留在 v2.0。这一套需要浮空受击、落地、重力曲线与空中相机，成本 40 小时起，放进当前范围会挤掉 Boss 的实现时间。

第二名角色、多敌人与小怪 AI 不做；跑图关卡与训练场不做；完整 GAS、C++ 与行为树不做。

范围收窄的目的是把时间全部压在战斗手感上。任何新想法都需要说明它挤掉哪一个里程碑。

11.4 尚未进入规格的待做项

韧性条随时间回升。它用于抑制纯观望的打法，但会连带影响破甲节奏与 TTK，需要重新核算一轮数值之后才能进规格。